SIMATS ENGINEERING
Engineer to Excel

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Capstone Project Title

Presented By

Student Name(s)	:	DHAARANEE SRI (1919260002) SUBASH SURYA N (1924260164) DURGA DEVI.G (1921260128) PADMAPRIYAN M (1924260030) SAINDHAVHI JS (1911260119)
Registration Number	:	
Course Name and Code	:	**CSA0803 - PYTHON PROGRAMMING**
Supervisor	:	
Date	:	

SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Introduction

- **Digital Transformation in Academics:** The rapid shift towards computer-based testing requires scalable, error-free assessment platforms.
- **Overview:** A centralized Python-based platform designed to create, conduct, and automatically evaluate objective and multiple-choice assessments.
- **Instant Feedback Mechanism:** Eliminates delays in score generation through real-time auto-grading algorithms.
- **Core Philosophy:** Reduces logistical overhead, saves paper, and delivers an unbiased evaluation environment.

SIMATS ENGINEERING
Engineer to Excel

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Problem Statement

- **Time-Consuming Manual Grading:** Traditional paper-based exams require extensive manual grading, leading to human error and delays.
- **Resource Inefficiency:** High consumption of physical resources (paper, printing, invigilation coordination).
- **Security & Cheating Risks:** Challenges in maintaining question randomization and prevent unauthorized access during tests.
- **Data Inconsistency:** Difficulties in securely storing, tracking, and retrieving historical student test records.

Objectives of the Project

- **Automate Assessment:** Develop an algorithmic auto-evaluation pipeline for instant result generation.
- **Role-Based Access:** Provide distinct panels with proper authentication for **Administrators/Teachers** and **Students**.
- **Question Bank Management:** Enable teachers to add, modify, categorize, and randomize questions.
- **Performance Analytics:** Automatically generate scorecards and performance summaries for students and faculty.
- **Integrity & Time Limits:** Implement real-time countdown timers and auto-submission upon timer expiration.

Scope of the Project

In-Scope:

- User registration and role-based login (Admin/Teacher & Student).
- Timed MCQ tests with automated randomized question ordering.
- Instant auto-grading against standard answer keys.
- Exporting results/analytics to CSV/PDF or dashboard viewing.

Out-of-Scope (Future Enhancements):

- Advanced subjective long-answer grading using NLP.
- AI-based remote webcam proctoring and biometric identity verification.

Related Work

Previous Systems & Technologies:

- *System 1:* **Native Desktop GUI Application:** Built using Python's tkinter library to eliminate browser dependencies, external hosting costs, and network loading delays.
- *System 2:* **Lightweight & Standalone execution:** Runs directly on standard desktop environments with minimal dependencies and fast GUI rendering.
- *System 3:* **Structured Test & Timer Management:** Features direct event-driven UI updates for precise exam time tracking, dynamic question rendering, and real-time score processing.

.

Related Work

Identified Gaps to fill:

- Need for lightweight, rapid Python implementation with structured database integration.
- Better randomized question distribution to minimize peer-to-peer malpractice.
- Lack of instant visual analytics and report generation for students.

.

Methodology / System Design

1. Deployment & Initialization:

- Extract the project .zip file and run main.py using Python runtime.
- System initializes into a desktop-native Tkinter GUI **Start Exam** landing interface.

2. Exam Session & Timed Execution:

- User taps **Start Exam** →Triggers the active **10-minute countdown timer** using Python event loops (after method).
- Dynamic multiple-choice questions (MCQs) are displayed sequentially using Tkinter widget components.

Methodology / System Design

3. Response Capture & Auto-Evaluation:

- Candidate selects responses for the MCQ set.
- System automatically compares selected choices against stored answer keys upon manual submission or timer expiration.

4. Result Generation & Retake:

- **Instant Result Page:** Displays total score, calculated percentage, and performance summary
- **Retake Mechanism:** Enables candidates to reset the session and restart the examination cleanly..

Methodology / System Design

Extract ZIP & Run main.py

↓

Tkinter Main Window / Start Screen

↓

[Start Timer (10 Min) & Load MCQs]

↓

Record Candidate Answers

↓

Submit / Auto-Submit on Timeout

↓

Auto-Evaluation Engine

↓

[Display Score, % & Retake Option]

Module Description

User GUI & Interface Module: Renders the main window using Python Tkinter components (Labels, Buttons, Radiobuttons) for seamless exam navigation

Question Data Module: Manages the structured list of multiple-choice questions, options, and correct answers directly within Python data structures.

Timer & Session Engine Module: Controls the 10-minute countdown using Python event loops (after method) and handles candidate selection tracking.

Auto-Evaluation & Results Module: Computes total scores and percentages upon submission or timeout, presenting the final feedback and retake option dynamically.

Expected Outcome

- **Automated Result Pipeline:** 100% reduction in grading turnaround time with zero human calculation errors.
- **Comprehensive Dashboards:** Instant student performance summaries with percentage breakdown and answer reviews.
- **High Efficiency & Scalability:** Lightweight Python architecture capable of managing multiple exams simultaneously.
- **Data Portability:** Clean export of class grade sheets in structured formats (CSV/Database tables).

Tools and Technologies

Frontend & Structure:

- . **Python Tkinter:** Serves as the primary graphical framework to build native desktop GUI windows, frames, buttons, labels, and radio buttons

- **Tkinter Canvas / Geometry Managers:** Handles dynamic layout positioning and visual formatting natively within the desktop application.

Core Logic & Scripting:

Python: Powers application initialization, active 10-minute timer threads/event loops, MCQ state tracking, dynamic score calculation, and exam reset logic

Tools and Technologies

Execution & Deployment Environment:

- **Python Runtime Environment:** Executes the application locally using Python, rendering the native graphical interface directly without requiring external software dependencies.

- **Distribution:** Packaged as a portable standalone .zip archive containing main.py for easy cross-platform desktop deployment and local execution.

Work Plan / Timeline

Day	Phase	Key Deliverables
Day 1	Problem Identification & Requirements	Scope definition, mapping exam parameters, and Python Tkinter GUI environment setup.
Day 2	System Architecture & Database Design	Designing Tkinter window layouts, dynamic GUI wireframes, and data structures for questions
Day 3	Backend Logic & Question Engine	Implementing question loading, multiple-choice option rendering, and candidate response tracking
Day 4	GUI & Module Integration	Assembling Tkinter frames, dynamic page transitions, and integrating the 10-minute active timer using
Day 5	Auto-Evaluation & Reporting	Instant scoring, percentage calculation, and result/retake interface.
Day 6	Testing & Bug Fixing	Boundary testing for timer expiration, radio button selections, window resizing, and application state resets.
Day 7	Documentation & Final Presentation	Code cleanup in main.py, project packaging (.zip), final presentation, and report write-up.

Conclusion and Future Plan

Conclusion:

- The developed Python-based Online Examination System delivers a secure, streamlined, and automated testing experience.
- Significantly minimizes administrative overhead while ensuring transparent and instantaneous evaluation.

Conclusion and Future Plan

Future Scope:

- Integrating AI-based computer vision for automated facial and tab-switch proctoring.
- Integrating Natural Language Processing (NLP) models (e.g., BERT/Transformers) for semantic evaluation of subjective text answers.

Thank You